Architecture Writeup

DSCI 601 – Applied Data Science I

Piter Z. Garcia

April 18, 2026

1 Overview

This architecture writeup describes the implementation of the code review repository used for the DSCI 601 assignment. The repository is organized as a small, reproducible *Hybrid CRISP+ML* pipeline that mirrors a real machine-learning workflow while remaining fast enough for reviewers to install, run, test, and inspect locally. The main design goal was to create a reviewer-safe system with clear stage boundaries, deterministic behavior, and enough implementation detail that another party could recreate the workflow and reproduce the same outputs.

The system is structured around seven explicit lifecycle stages:

1. Data Collection
2. Data Cleaning
3. Data Preparation
4. Data Exploration / Visualization
5. Data Mining
6. Evaluation
7. Results Postprocessing / Visualization

These stages were chosen so the implementation matches both a recognized methodology and the assignment requirements, which ask for separable runnable code for data handling, modeling, exploration, and postprocessing.

2 Architecture and Layer Interactions

Figure 1 shows the overall system structure. The software is organized into four main layers: configuration, stage execution, orchestration, and outputs.

2.1 Configuration Layer

The configuration layer defines the parameters that control the entire run. In the implementation repository, this is handled by a typed configuration object loaded from a single JSON file. The configuration stores the random seed, record counts, train/test split, invalid-record frequency, missingness frequency, bias level, fairness threshold, and output directory.

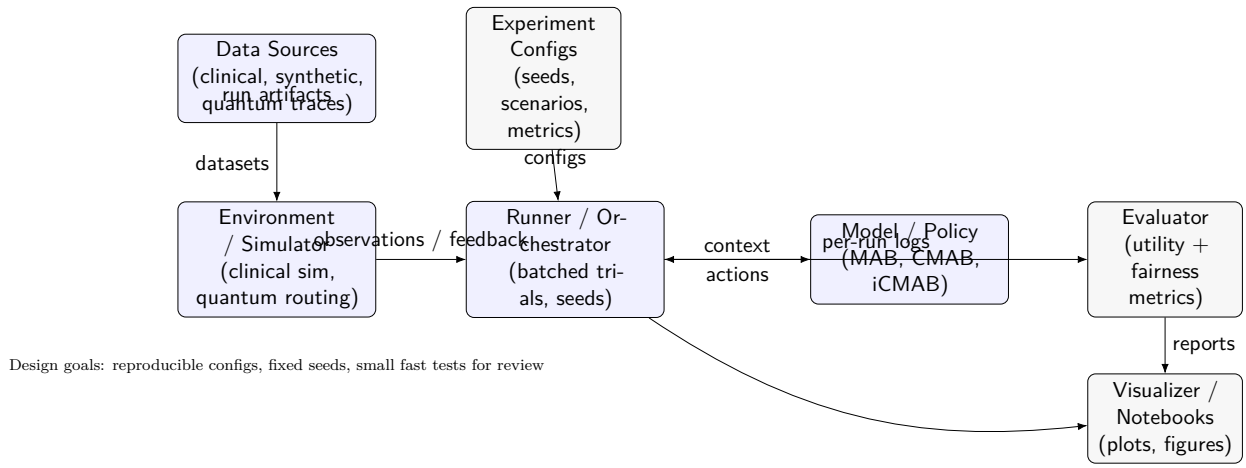

Figure 1: Architecture of the reviewer-safe CRISP+ML pipeline.

This layer was separated from the stage logic for two reasons. First, it keeps experiments reproducible: the same configuration and seed lead to the same output artifacts. Second, it makes the system easy to inspect and modify without changing source code. A reviewer can understand the workflow by reading one config file instead of tracing hard-coded constants across the repository.

2.2 Stage Layer

Each lifecycle step is implemented as its own concrete class under a shared `PipelineStage` interface. This makes the code strongly object-oriented and keeps stage responsibilities narrow. The stage implementations behave as follows:

- **Data Collection** generates a small synthetic dataset that preserves fairness-relevant structure such as group membership, label, and feature availability.
- **Data Cleaning** validates records, removes intentionally corrupted examples, and standardizes the remaining schema.
- **Data Preparation** converts cleaned records into model-ready train and test artifacts.
- **Data Exploration** creates lightweight summaries of dataset size, group distribution, and feature completeness.
- **Data Mining** runs the minimal modeling step on the prepared dataset and writes predictions.
- **Evaluation** computes utility and fairness-oriented metrics from predictions.
- **Results Postprocessing** turns evaluation summaries into reviewer- friendly output tables and simple figures.

Every stage reads only the artifacts it needs, writes to its own dedicated output directory, and returns a structured result object containing artifact paths and summary metadata. This design keeps the stages testable in isolation and reduces the risk of hidden side effects between steps.

2.3 Context and Orchestration Layer

The orchestration layer is responsible for connecting the individual stages together. A shared pipeline context object carries metadata, output locations, and stage results through the run so that stages do not need direct knowledge of each other's internal implementation. This context acts as a lightweight contract for information sharing.

The `ReviewPipeline` orchestrator builds the ordered list of stages and executes them in sequence. It can run the full workflow or stop after any named stage. At the end of execution, it writes a manifest file that records the active configuration, stage outputs, and stage summaries. This manifest becomes the single reproducibility record for a run.

2.4 Interface Layer

The user-facing interface is a command-line entry point with one command for each stage plus two orchestration commands: one for running the full pipeline and one for performing the local merge validation path. This layer was important for the assignment because the repository needed explicit runnable scripts rather than a single notebook. The command-line interface lets another party run individual phases independently or validate the full workflow end-to-end.

3 Framework and Component Choices

The architecture intentionally uses a very small technical stack. The pipeline is implemented in standard-library Python with a minimal package structure and `requirements.txt` as the reviewer-facing install path. This choice was made to reduce environmental friction during code review. Reviewers do not need to install a heavy machine-learning stack or resolve hidden dependencies just to run the repository.

JSON was selected for configuration and most structured artifacts because it is easy to inspect, easy to validate, and natively supported in Python. CSV was used for summary tables because it is a familiar exchange format for grading and quick inspection. SVG was used for lightweight visual outputs because it provides a clear figure artifact without requiring plotting libraries. The overall component selection favors reproducibility, transparency, and short setup time over raw performance.

4 Data Storage and Output Design

All artifacts are written beneath a stable output root defined by the active configuration. Within that root, each stage writes to its own numbered directory. This output scheme was chosen for traceability. A reviewer can inspect any stage without rerunning the full system, and each run naturally leaves behind a visible artifact trail.

The output structure is designed to answer three practical questions:

- What input configuration produced this run?
- What did each stage write?
- What metrics and final outputs were generated?

Those questions are answered by the combination of stage directories and the final manifest file. This layout also supports merge validation because the validation step can check for expected artifact existence in deterministic locations.

5 Reproducibility Path

Another party can recreate the system by setting up a fresh Python environment, installing the repository, and executing the pipeline with the provided fast review configuration. The expected workflow is:

1. install dependencies from `requirements.txt`,
2. run the full pipeline command with the provided configuration,
3. inspect the generated results directory,
4. run the merge-check command to execute the local validation path.

This path reproduces the same stage artifacts, metrics, summaries, and validation checks that the repository uses for review. Because the system is configuration- driven, deterministic, and documented through explicit stage boundaries, another party should be able to recreate both the software behavior and the presented results with minimal ambiguity.

6 Conclusion

The architecture emphasizes simplicity, determinism, and inspectability. Rather than optimizing for scale, the design optimizes for a clean code review experience: small inputs, explicit stages, typed configuration, structured outputs, and a repeatable validation path. This makes the system appropriate both for the DSCI 601 assignment and for future extension into larger reproducible workflows.